

Q.1) Consider the following function that takes reference to head of a Doubly Linked List as parameter. Assume that a node of doubly linked list has previous pointer as *prev* and next pointer as *next*.

```
void fun(struct node **head_ref)
{
    struct node *temp = NULL;
    struct node *current = *head_ref;

    while (current != NULL)
    {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }

    if(temp != NULL )
        *head_ref = temp->prev;
}
```

Assume that reference of head of following doubly linked list is passed to above function 1 <--> 2 <--> 3 <--> 4 <--> 5 <--> 6. What should be the modified linked list after the function call?

Answer: 6 5 4 3 2 1

<https://www.youtube.com/watch?v=wzogfnD1wrw>

Q.2) The following function reverse() is supposed to reverse a singly linked list. There is one line missing at the end of the function.

```
/* Link list node */
struct node
{
    int data;
    struct node* next;
};

/* head_ref is a double pointer which points to head (or start) pointer
of linked list */
static void reverse(struct node** head_ref)
{
    struct node* prev = NULL;
    struct node* current = *head_ref;
    struct node* next;
    while (current != NULL)
    {
        next = current->next;
```

```

    current->next = prev;
    prev = current;
    current = next;
}
/*ADD A STATEMENT HERE*/
}

```

What should be added in place of "/*ADD A STATEMENT HERE*/", so that the function correctly reverses a linked list.

Answer: *head_ref = prev;

Q3) Consider a singly linked list having n nodes. The data items $d_1, d_2, \dots, d_n$ are stored in the n nodes. Let Y be a pointer to the j th node ($1 \leq j \leq n$) in which d_j is stored. A new data item d stored in a node with address Y is to be inserted. Give an algorithm to insert d into the list to obtain a list having items $d_1, d_2, \dots, d_{j-1}, d, d_j, \dots, d_n$ in that order without using the header.

Solution:

Algorithm 1 insert_mid()

```

1: create newnode
2: newnode->next=y->next
3: newnode->data=y->data          /*which is dj*/
4: y->next=newnode
5: y->data=d

```

Explanation:

Since we didn't have the address of node which is previous to Y .

So insert a new node after Y .

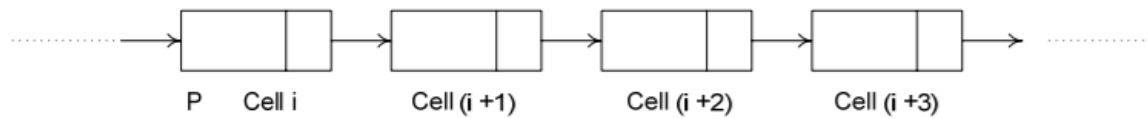
And copy the data of Y to new node and modify data field of Y to d .

Hence we get required sequence of data as $d_1, d_2, \dots, d_{j-1}, d, d_j, \dots, d_n$

<https://www.youtube.com/watch?v=xoXfoJOZOr4&t=3121s>

Q4)

Let p be a pointer as shown in the figure in a singly linked list.



What do the following assignment statements achieve?

$q := p \rightarrow \text{next}$

$p \rightarrow \text{next} := q \rightarrow \text{next}$

$q \rightarrow \text{next} := (q \rightarrow \text{next}) \rightarrow \text{next}$

$(p \rightarrow \text{next}) \rightarrow \text{next} := q$

Solution: <https://www.youtube.com/watch?v=xoXfoJOZOr4&t=3121s>

Q5) Consider the following piece of ‘C’ code fragment that removes duplicates from an ordered list of integers.

```
Node *remove-duplicates(Node *head, int *j)
{
    Node *t1, *t2;
    *j=0;
    t1 = head;
    if (t1 != NULL) t2 = t1 →next;
    else return head;
    *j = 1;
    if(t2 == NULL)
        return head;
    while t2 != NULL)
    {
        if (t1.val != t2.val) -----→ (S1)
        {
```

```

        (*j)++; t1 -> next = t2; t1 = t2: -----> (S2)
    }
    t2 = t2 -> next;
}
t1 -> next = NULL;
return head;
}

```

Assume the list contains n elements ($n \geq 2$) in the following questions.

- How many times is the comparison in statement *S1* made?
- What is the minimum and the maximum number of times statements marked *S2* get executed?
- What is the significance of the value in the integer pointed to by j when the function completes?

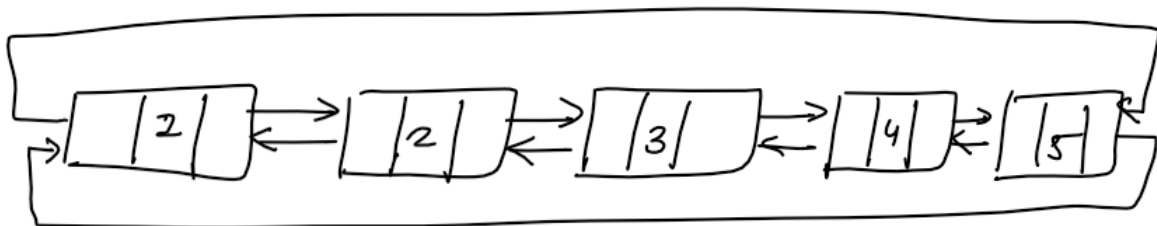
Answer:

Explanation:

- $n-1$ times, since comparison is pairwise for n elements.
- maximum : $n-1$ for all distinct elements, minimum: 0 for all same elements.
- (C). j keeps count of distinct nodes in the list.

<https://www.youtube.com/watch?v=xoXfoJOZOr4&t=3121s>

Q6) The following figure depicts the initial configuration of doubly linked list.



What would be the output after the following sequences of steps?

- List->next->next->next = list->prev
- List->prev->prev->prev = list->next->next->next->prev
- List->next->next->next->prev = list->prev->prev->prev
- Printf("%d", list->prev->prev->prev->data)

- a) 1 b) 2 c) 3 d) 4

Solution: <https://www.youtube.com/watch?v=S4mkHIGEohM>